

A imagem é composta por **três blocos visuais simples**:

1. **Qualidade:**
    a. **Software Robusto e Confiável:** Foco em entregas seguras.
    b. **Código Limpo:** Facilidade de manutenção.
    c. **Menos Defeitos (Bugs):** Redução de erros no sistema.
    d. **Foco no Usuário:** Boa experiência para o cliente final.
2. **DoD (Definition of Done):**
    a. **Critérios Concluídos:** Atendimento aos requisitos.
    b. **Testes Aprovados:** Testes de unidade e integração bem-sucedidos.
    c. **Revisão de Código:** Código verificado pelo time.
    d. **Publicação em Teste:** Sistema pronto em ambiente de homologação.
3. **CI / Refatoração:**
    a. **Integração Contínua (CI):** Automação de compilação e testes.
    b. **Refatoração Frequente:** Melhoria contínua da estrutura do código sem alterar seu funcionamento externo.

1. Introdução à Qualidade na Agilidade

- **O Paradigma da Qualidade:** No modelo tradicional de desenvolvimento, a qualidade era tratada como uma etapa isolada e burocrática que acontecia apenas no final do projeto (uma fase inteira dedicada exclusivamente a testes). Na agilidade, a qualidade **não é uma fase final, mas sim uma prática contínua** embutida em todas as etapas do processo de desenvolvimento.

- **O Foco na Prevenção:** Em vez de apenas caçar bugs depois que o código está pronto, a agilidade utiliza técnicas de engenharia e alinhamento de critérios para garantir que o software nasça correto, limpo e testável desde o primeiro dia.

2. Definition of Done (DoD - Definição de Pronto)

- **O que é:** É um acordo explícito e compartilhado por toda a equipe sobre o que significa uma tarefa estar **realmente finalizada**. Enquanto o *Definition of Ready* (DoR) define quando uma tarefa está pronta para *começar*, a DoD define quando ela pode ser considerada 100% concluída.
- **Exemplos de critérios comuns na DoD:**
    - O código foi escrito e revisado por outro colega (*Code Review*).
    - Todos os testes automatizados (unitários e de integração) passaram com sucesso.
    - O código foi integrado ao repositório principal (*CI*).
    - A documentação técnica foi atualizada.
    - O item foi testado e aprovado em ambiente de homologação.
- **Para que serve:** Evita a famosa ilusão do desenvolvedor que diz *"meu código está pronto, só falta testar"*. Na agilidade, se não passou por todos os critérios da DoD, a funcionalidade **não está pronta** e não pode ser entregue.

3. Testes Automatizados

- **O que são:** São scripts de software criados especificamente para testar partes do sistema de forma automática, rodando em frações de segundo por meio de comandos ou ferramentas de integração.
- **Por que são vitais na agilidade:** Como os ciclos de entrega (*sprints*) são extremamente curtos, seria humanamente impossível testar todo o sistema manualmente a cada nova alteração sem perder prazos e deixar escapar bugs críticos.
- **Principais tipos básicos:**
    - **Testes Unitários:** Testam a menor parte isolada do código (como uma função ou método matemático específico).
    - **Testes de Integração:** Verificam se diferentes módulos ou componentes do sistema conversam e funcionam bem juntos.
    - **Testes de Aceitação / Sistema:** Validam se o comportamento geral do sistema atende às regras de negócio esperadas pelo usuário.

4. Integração Contínua (CI - Continuous Integration)

- **O que é:** É uma prática de desenvolvimento de software em que os membros da equipe integram e juntam seus códigos em um repositório central compartilhado várias vezes ao dia (ou pelo menos diariamente).
- **Como funciona na prática:** Cada vez que um desenvolvedor envia (*push*) seu código novo para o repositório central, um servidor automatizado dispara uma série de rotinas para compilar o sistema e rodar todos os testes automatizados instantaneamente.

- **Para que serve:** Evita o pesadelo clássico do "inferno da integração", que ocorria no modelo tradicional quando os programadores passavam meses codificando isoladamente e tentavam juntar tudo no último dia, gerando conflitos gigantescos e quebrando o sistema inteiro. A CI avisa a equipe imediatamente se o código novo quebrou alguma parte do sistema existente.

5. Refatoração

- **O que é:** É o processo de **reestruturar e limpar um código fonte existente** sem alterar o seu comportamento externo ou as funcionalidades que ele entrega ao usuário.
- **O objetivo principal:** Melhorar a legibilidade do código, reduzir a complexidade lógica, eliminar duplicações e facilitar futuras manutenções.
- **Analogia simples:** É como arrumar, limpar e organizar uma gaveta de ferramentas bagunçada para que você consiga encontrar e usar os utensílios muito mais rápido na próxima vez, sem mexer no que as ferramentas fazem.
- **Relação com a agilidade:** O código em projetos ágeis evolui de forma orgânica. Refatorar constantemente garante que o sistema continue flexível e saudável ao longo do tempo.

6. Dívida Técnica (*Technical Debt*)

- **O que é:** É um conceito cunhado por Ward Cunningham que descreve o custo implícito de escolher uma solução de desenvolvimento rápida, "gambiarra" ou de baixa qualidade a curto prazo, em vez de aplicar a solução arquitetural correta e limpa.
- **Por que ela surge:** Muitas vezes, para entregar uma funcionalidade correndo e aproveitar uma oportunidade de mercado (*time-to-market*), a equipe toma um atalho no código.
- **O Perigo do Acúmulo:** Assim como uma dívida financeira com juros, se a equipe acumula muita dívida técnica sem reservação de tempo para limpá-la (refatoração), o código vai se tornando podre, frágil e lento de modificar. Com o tempo, a equipe passa a gastar mais tempo consertando problemas antigos do que criando novos valores para o cliente.

FONTES UTILIZADAS

- https://martinfowler.com/articles/continuousIntegration.html
- https://martinfowler.com/bliki/TechnicalDebt.html
- https://refactoring.guru/refactoring
- https://www.scrum.org/resources/blog/done-understanding-definition-done
- https://engsoftmoderna.info/cap9.html